Chapter 1: Computer Abstractions and Technology

(*) Performance কাদের উপর depend করে?

1) Algorithm / Original Program

Algorithm simple হলে performance ভালো হবে, time কম লাগবে। Algorithm complex হলে performance খারাপ হবে, অর্থাৎ time বেশি লাগবে।

(ii) Programming language, compiler, architecture

Main focus: Assembly language

Software used to translate the program into machine instructions.

একটা code Python and C তে করা হলে Python এর ক্ষেত্রে time বেশি লাগবে। কারণ এটি human-like বেশি। অন্যদিকে C হচ্ছে assembly language এর closer, time খুব কম হলেও Python থেকে কম time এ compile করতে পারবে। So, programming language এর উপরেও performance depend করে।

(iii) Processor and memory system

Intel pentium এর সাথে intel core i3, i7, i9 এর comparison করবো। প্রত্যেকটা chipwise performance better হচ্ছে।

iv) I/O system (including OS)

I/O system থেকে কিভাবে data actual microprocessor এর কাছে যায়

* Performance এর software component কি কি?

i) Algorithm

ii) Programming language, compiler and architecture

* Performance এর hardware component কি কি?

i) Processor and memory system

ii) I/O system (Hardware and OS)

Application software: আমরা যে software use করি, সেগুলো application software, যা খুবই high level language (python, C, Java).

System software:

Compiler: High level language to machine code এ translate করে।

Operating system: HLL কে normal code এ রূপান্তর করে

Hardware: Then hardware normal code নিয়ে কাজ করে।

High level code

↓ compiler

Assembly language

↓ Assembler

Binary digit

(*) five classic components of a computer:

i) Input

ii) Output

iii) Memory

iv) Datapath (The part of the computer where data is processed)

v) Control

Datapath: কোন path দিয়ে data আদান-প্রদান করছে।

control: (CPU) CPU control করে datapath এ কোন কাজ হচ্ছে

Memory: variable কে memory address store করে রাখতে পারে।

* How the software or hardware component affects performance ?

Algorithm: Determine both the number of source-level statement and the number of I/O operations executed.

Programming language, compiler and architecture: Determine the number of computer instructions for each source-level statement.

Processor and memory system: Determines how fast instructions can be executed.

I/O system (Hardware and OS): Determines how fast I/O operations can be executed.

Seven Great Ideas in Computer Architecture

* Use Abstraction to simplify Design: It divide systems into layers to manage complexity and hiding the lower level details.

Example: A Python programmer doesn't need to know how the CPU executes instructions or how memory

is managed - they work with high-level commands, while the interpreter handles the details.

Another example: You drive a car without knowing how the engine works.

* Make the common case fast: Making the common case fast will tend to enhance performance better than optimizing the rare case. Ironically, the common case is often simpler than the rare case and hence is usually easier to enhance. (Optimizing frequently performed operations)

Example: I've 10 instruction (6 are addition, 4 are random) Since addition happens more frequently (6 out of 10), we can optimize the hardware or software to make addition super fast, even if the other instructions are a bit slower.

Performance via Parallelism: Running tasks simultaneously to boost speed.

Example: Me and my friend cook together - one cuts vegetables, the other boils water. Done faster.

In computer: Multi-core CPUs do multiple tasks at the same time

Performance via pipelining: A way to achieve parallelism.

Example: A CPU processes one instruction in multiple stages.

Parallelism: many workers, each doing a full job

Pipelining: one worker, doing parts of many jobs at once

---

* Performance via Prediction: Anticipate future data or values to avoid delays.

Example: My phone autocompletes my typing ("Hello, h → how are you?"). It predicts and saves time. (statistically, the guesses are often correct and the cost of recovering from a wrong guess is relatively low).

Hierarchy of memories: Organizing memory into levels (registers, cache, RAM, storage) based on speed and size.

Example: I keep important things in my pocket,

less important ones in my backpack and the rest at home.

In computers: Fast cache, medium RAM, slow hard disk. (The memory hierarchy helps the computer work faster by keeping the most used data in the fastest memory and less-used data in slower storage.)

* **Dependability** via redundancy: Adding extra components ensures reliability by compensating for failures in critical hardware or data. (Have a backup - just in case something breaks).

Example: Dual power supplies

Servers often have two power supplies - If one fails, the other takes over instantly and the system keeps running without interruption.

Computers need to be both fast and reliable. Since any hardware can break, we make systems reliable by adding extra components that can take over if something breaks and to help spot any problem.

Response time and Throughput

Response time : 1 টা tasks করতে কত time লাগে

Throughput : 1 unit time এ কতটুকু task করতে পারে

1 task → x time → response time

1 time → x task → Throughput.

Relative Performance

$$\text{Performance}_x = \frac{1}{\text{Execution time}_x}$$

$$P = \frac{1}{E}$$

$$\text{Performance} = \frac{\text{clock Rate}}{\text{CPI}}$$

Execution time / Response time / CPU time

Response time: The time between the start and completion of a task.

Throughput: Total work done per unit time.

Measuring Execution Time

Elapsed time: Elapsed time is the total time taken from the start to the completion of a task or program. It includes all types of delays and overhead, not just CPU work.

CPU time: The actual time the CPU spends computing for a specific task. It does not include time spent waiting for I/O or non time when the CPU is working on other processes.

clock period: (clock cycle time) single clock cycle ছাড়া সময়

clock period is the time duration of one cycle of a computer's clock signal (T)

clock frequency (f): (clock Rate) clock frequency is the number of clock cycles per second in a computer system.

$$f = \frac{1}{T}$$

CPU Time = CPU clock cycles × clock cycle time

$$= \frac{\text{CPU clock cycles}}{\text{clock rate}}$$

clock cycle = Instruction count × cycles per instruction

CPU time = instruction count × CPI × clock cycle time

$$= \frac{\text{instruction count} \times \text{CPI}}{\text{clock rate}}$$

Power Trends : যতবারি voltage দিচ্ছি, তত performance বাড়তেছে, at one point, voltage দেয়ার পর performance low হওয়া শুরু হয়েছে। এর কারণ, voltage দেয়ার পর material বার বার vibrate করলে, by laws of thermodynamics এখানে heat generate হবেই। এই heat কে সরাতে না পারলে, voltage দেয়ার পর- at one point performance low হয়। This concept is called power trend.

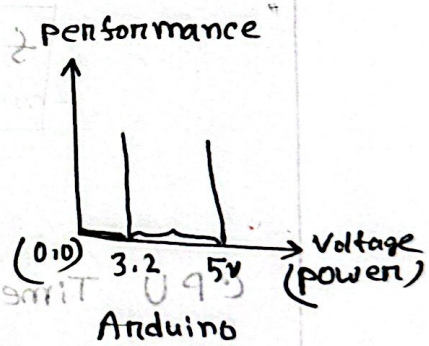
Power = Capacitive load x voltage^2 x Frequency

***

* What is the power wall ?

i) we can't reduce voltage further

ii) we can't remove more heat



1st power wall: এই wall এর নিচে power reduce করি, তাহলে device কেই হারিয়ে ফেলতে।

2nd power wall: এর বেশি power বাড়াতে থাকি, তাহলে আমার heat generate হত, heat remove না করে, কাজ করতে পারতেছি না।

SPEC Ratio : $\frac{\text{reference time}}{\text{execution time}}$

SPEC CPU Benchmark is a standardized set of tests used to measure and compare the performance of computer processors.

	Ref	exec
A	1432	698
B	1703	987
C	2939	1718

$A = \frac{1432}{698}$

$B = \frac{1703}{987}$

$C = \frac{2939}{1718}$

Geometric Mean ↗ 1/যে কয়টা term

So, $(A \times B \times C)^{1/3}$

$= \sqrt[3]{A \times B \times C}$

***

Amdahl's Law: This law helps us to understand the overall performance improvement gained by optimizing a single part of a system

Example: multiply accounts for 80s/100s

How much improvement in multiply performance to get 5X overall?

100 → execution time

- 80 sec multiplication (serial part)
- 20 sec (all other) (parallel part)

$P = \frac{1}{E}$

$\Rightarrow 5P = \frac{5}{E}$

100/5 = 20 sec

can't be done (20 sec লাগে all other করতে, so 20 sec এ program run করানো যাবে না)

* 2 X overall

$\frac{100}{2} = 50 sec$

50 sec
↓ ↓
30sec (multi) 20sec (other)

80 → 30 sec

multiplication এর performance বাড়াতে হবে

$$T_{improved} = \frac{T_{affected}}{improvement\ factor} + T_{unaffected}$$

$$50 = \frac{80}{n} + 20$$

* কতটুকু better করলে মান performance?

MIPS: Millions of Instructions Per Second

2 MIPS → 2 million per second

CPI = 2 → ins count

1 sec ⇌ 2 million

1 ins taken 2 pulse

∴ 2 " " 2×2 = 4 million

1 sec ⇌ 4×10^6 pulse

$4 \times 10^6\ sec^{-1} = 4 \times 10^6\ Hz$

$= 4\ MHz$

Yield: The percentage of good dies from the total number of dies on the wafer.

Die: The individual rectangular sections that are cut from a wafer, more informally known as chip.

$$\text{Dies per wafer} \approx \frac{\text{Wafer area}}{\text{die area}}$$

$$\text{Cost per die} = \frac{\text{Cost per wafer}}{\text{Dies per wafer} \times \text{yield}}$$

$$\text{Yield} = \frac{1}{(1 + (\text{Defects per area} \times \text{Die area}))^{N}}$$

As the shape of a wafer is round, so wafer area = πr^2

Die can be rectangular or square shape.

Area = Height × Width.

* Dies per wafer ≈ Wafer area / Die area. Explain why there is '≈' not '=' ?

- The formula is an approximation to give a quick estimate. The real number of dies per wafer is always a bit lower due to: i) edge effects, (ii) scribe lines; (iii) Defects/yield issues. These factors make the actual die count slightly less than the simple area ratio. So, we use '≈' instead of '='.

Benchmark is a process of measuring or comparing the performance of the system or a process against the standard or the best performing (counter parts or reference computer).

* Perlbench run ~~করে~~ instruction count ~~$\times 10^9$~~
$= 2684 \times 10^9$, CPI = 0.42, clock cycle time $= 0.556 \times 10^{-9}$.

Execution time $= 2684 \times 10^9 \times 0.42 \times 0.556 \times 10^{-9}$

$= 627$

Reference time = 1774

SPEC Ratio = $\frac{\text{Reference time}}{\text{Execution time}} = 2.83$

Execution time < Reference time

∴ So faster

* SPEC ratio of Perlbench is 2.83. What's the meaning of it?

- Perlbench program runs almost 2.83 times faster in my computer, than the reference computer.

Benchmark 0. something ~~হলে~~, Program is slower.

Geometric Performance 2.36, what does it mean?

- Average performance 2.36 times faster in my computer than the reference computer.

* A program P has two operations. Between them, the serial operations take 80s and parallel operations take 50sec. If you want to run the program in 110sec, what should be the improvement factor of the parallel operation

$$T_{improved} = \frac{T_{assected}}{n} + T_{unassected}$$

$$\Rightarrow 110 = \frac{50}{n} + 80$$

$$\Rightarrow n = 5/3$$

* Computer A executes 15 instructions in 3sec. Find MIPS.

↓ million of instructions executed per second

3sec $\rightarrow$ 15

$\therefore$ 1 sec $\rightarrow \frac{15}{3} = 5$

1000000 ins - 1 m

$\therefore$ 1 ins - $\frac{1}{1000000}$ m

$\therefore$ 5 ins - $\frac{5}{1000000}$ m = 5×10^{-6} M

Practice Problem 1:

$IC = 4.5 \times 10^{12}$

Clock Rate = 2.5 GHz = 2.5×10^9 Hz

	A	B	C	D
IC	$4.5\times10^{12} \times 0.1$ $= 4.5 \times 10^{11}$	$4.5\times10^{12}\times0.2$ $= 9\times10^{11}$	$4.5\times10^{12}\times0.5$ $= 2.25\times10^{12}$	$4.5 \times 10^{12} \times 0.2$ $= 9\times10^{11}$
CPI	1	2	3	3

$$\text{CPU Time} = \frac{\text{Instruction count} \times \text{CPI}}{\text{Clock Rate}}$$

সবগুলো instructions এর CPI same না হলে, we need to use Average CPI.

$$\text{Average CPI} = \frac{(4.5\times10^{11}\times1)+(9\times10^{11}\times2)+(2.25\times10^{12}\times3)+(9\times10^{11}\times3)}{4.5\times10^{12}}$$

$$= 2.6$$

$$\text{CPU Time} = \frac{4.5\times10^{12} \times 2.6}{2.5\times10^9}$$

$$= 4680 \text{ sec}$$

$$IC_{new} = 4.5 \times 10^{12} + (4.5 \times 10^{12} \times 0.1)$$
$$= 4.95 \times 10^{12}$$

New Average CPI $= 2.6 + (2.6 \times 0.05)$

$= 2.73$

$$\text{CPU Time}_{new} = \frac{IC_{new} \times CPI_{new}}{\text{clock rate}}$$

$$= \frac{4.95 \times 10^{12} \times 2.73}{2.5 \times 10^{9}}$$

$= 5405.4$ sec

Increase CPU time $= (5405.4 - 4680)$ sec

$= 725.4$ sec

Practice Problem 3:

> Clock cycle time / clock duration / Time taken to complete a cycle

$CCT_{PA} = 4$ ns $\qquad$ $C.C.T_{P2} = 3.5$ ns

$CPI_{PA} = 0.7$ $\qquad$ Avg $CPI_{P2} = 0.7$

> While comparing, you must use the same program.

CPU time = IC x CPI x Clock cycle time

$$\frac{\text{CPU time}_{PA}}{\text{CPU time}_{P2}} = \frac{IC \times CPI_{PA} \times CCT_{PA}}{IC \times CPI_{P2} \times CCT_{P2}}$$

$$= \frac{IC \times 0.7 \times 4}{IC \times 0.7 \times 3.5}$$

$$\Rightarrow \frac{\text{CPU time}_{PA}}{\text{CPU time}_{P2}} = \frac{4}{3.5} = 1.142$$

$$\Rightarrow \text{CPU Time}_{PA} = 1.142 \times \text{CPU Time}_{P2}$$

Processor P2 is 1.142 times faster than Processor PA.

Practice Problem 3

CPU time = 5 sec

Clock cycle = Instruction count × CPI

$= 25M \times 2 = 50M = 50 \times 10^6$

5 MIPS ⇒

1 s — 5 ms

∴ 5 s — (5×5) = 25M

Practice Problem 4

a) CPU time = 250 s

Add	Sub	left shift	Mis
70	85	40	250 - 195 = 55

70 + 85 + 40 = 195

New time = (70 X 0.8) + 85 + 40 + 55

= 236 sec

b)

T_{new} = 250 X 0.8 = 200 sec

70 + 85 + 55 = 210 sec

210 > 200

even If we consider left shift as 0, we cannot get 200 sec.

So, the total time cannit be reduced by 20% by reducing only the time for left-shift instructions.